

Практическая работа №2.

Работа с данными для хранения больших данных. Parquet, Avro.

1. Цель и результаты.

Цель: познакомить студентов с форматами хранения больших данных Parquet и Avro, их особенностями, плюсами/минусами и базовыми операциями чтения/записи в Python (pandas / PySpark). Форматы широко используются для эффективного хранения и аналитики: Parquet как колонко-ориентированный формат, Avro как формат сериализации с поддержкой эволюции схем.

Ожидаемые результаты: студент умеет объяснить, чем Parquet отличается от Avro и от CSV; записывать и читать данные в Parquet и Avro, работать со схемой Avro; выбирать подходящий формат под тип нагрузки (аналитика и потоковая обработка).

2. Исходные данные.

Требования к окружению:

Python 3.10+

Библиотеки: pandas, pyarrow, fastavro (или avro), при желании pyspark.

Подготовленный CSV-файл на 100k+ строк (например, лог продаж, клики, измерения сенсоров).

Пример датасета: data/sales.csv с колонками order_id, user_id, item_id, price, quantity, ts.

3. Задания.

Задание 1. Изучить недостатки CSV для больших данных.

1. Загрузить CSV-файл в pandas, измерить:
время чтения;
размер файла на диске.
2. Задать вопросы:
какие проблемы возникают при хранении больших CSV (объём, отсутствие схемы, типы, отсутствие сжатия, отсутствие эволюции схемы и т.п.)?

Краткий ответ оформить текстом (3–5 предложений).

Задание 2. Работа с Parquet.

1. Прочитать тот же CSV в pandas, привести типы (например, `ts` → `datetime`, `price` → `float`).
2. Сохранить фрейм в формате Parquet с использованием `pyarrow` и сжатия `snappy` или `gzip`.
3. Сравнить:
размер CSV и Parquet-файла;
время чтения CSV и Parquet.
4. Продемонстрировать выборочных столбцов: прочитать только колонки `user_id`, `price` из Parquet (`column pruning`) и измерить время / объём данных.

Кратко ответить:

почему Parquet читает быстрее при аналитических запросах, особенно на подмножестве колонок;

для каких задач Parquet особенно полезен (DWH, аналитические запросы, datalake).

Дополнительное задание. Вариант через PySpark: считать CSV как DataFrame, сохранить в Parquet, показать explain простого SQL-запроса и predicate pushdown (фильтрация по колонкам). (explain в Spark — это команда/метод, который показывает план выполнения запроса: какие шаги Spark сделает, чтобы получить результат, predicate pushdown (иногда говорят filter pushdown) — это оптимизация, при которой условия фильтрации выполняются как можно ближе к источнику данных, а не после того, как всё уже прочитано в Spark.)

Задание 3. Работа с Avro и схемой.

1. Определить Avro-схему для того же набора полей в JSON-виде: типы, значения по умолчанию и т.п.
2. Записать данные в Avro-файл с помощью fastavro или другой библиотеки.
3. Прочитать Avro-файл и убедиться, что типы корректны.

Далее показать эволюцию схемы (простая версия):

4. Создать новую схему:
добавить новое поле с значением по умолчанию;
удалить необязательное поле.

5. Прочитать старые данные с новой схемой, показать, как подставляются значения по умолчанию и как Avro поддерживает forward/backward-совместимость. (Backward-совместимость (обратная) - новая версия схемы может читать данные, записанные по старой схеме. Forward-совместимость (прямая) - старая версия схемы может читать данные, записанные по новой схеме.)

Пояснить, в чём преимущество Avro-подхода к эволюции схем, особенно в потоковых системах и при использовании registry (Kafka, Hadoop-экосистема).

Задание 4. Parquet vs Avro: сравнительный анализ.

Заполнить таблицу (в отчёте), основываясь на экспериментах и кратком теоретическом обзоре:

Критерий	Parquet	Avro
Модель хранения		
Основной сценарий применения		
Сжатие и скорость аналитических запросов		
Эволюция схем		
Типичные интеграции		

Заключение: студент формулирует 5–7 предложений, в каких частях современной архитектуры данных целесообразно использовать Avro, а в каких — Parquet (например: ingestion в Avro, последующее сохранение в Parquet для аналитики).

4. Контрольные вопросы.

1. Почему Parquet считают «колоночным» форматом и как это влияет на производительность аналитических запросов?
2. В чём ключевое преимущество Avro с точки зрения эволюции схем данных?
3. В каких сценариях Parquet будет хуже, чем Avro, и наоборот?
4. Почему в гибридных архитектурах часто используют Avro для ingestion и Parquet для долговременного хранения?

5. Библиографический список.

1. <https://www.databricks.com/glossary/what-is-parquet>
2. <https://www.dremio.com/wiki/avro-format/>
3. <https://airbyte.com/data-engineering-resources/parquet-vs-avro>
4. <https://sqream.com/blog/a-detailed-introduction-to-the-avro-data-format/>
5. <https://www.datacamp.com/blog/avro-vs-parquet>
6. <https://www.datacamp.com/tutorial/apache-parquet>
7. <https://parquet.apache.org/docs/file-format/>
8. <https://cologix.com/blog/parquet-file-format/>
9. <https://motherduck.com/learn-more/why-choose-parquet-table-file-format/>

10. <https://www.qlik.com/blog/what-is-the-parquet-file-format-use-cases-and-benefits>
11. https://docs.oracle.com/cd/E26161_02/html/GettingStartedGuide/schemaevolution.html
12. <https://clickhouse.com/resources/engineering/avro-vs-parquet>
13. <https://parquet.apache.org/docs/overview/>
14. <https://stackoverflow.com/questions/76100579/schema-evolution-in-avro>
15. https://en.wikipedia.org/wiki/Apache_Parquet